DEPHIDES

Deep Learning Based Phishing URL Detection System

DA-221 Artificial Intelligence Course Project

P. Sai Deva Charan Roll: 240150024

Menni Charan Sree Teja Roll: 240150018

Lalit Santosh Deshmane Roll: 240150044

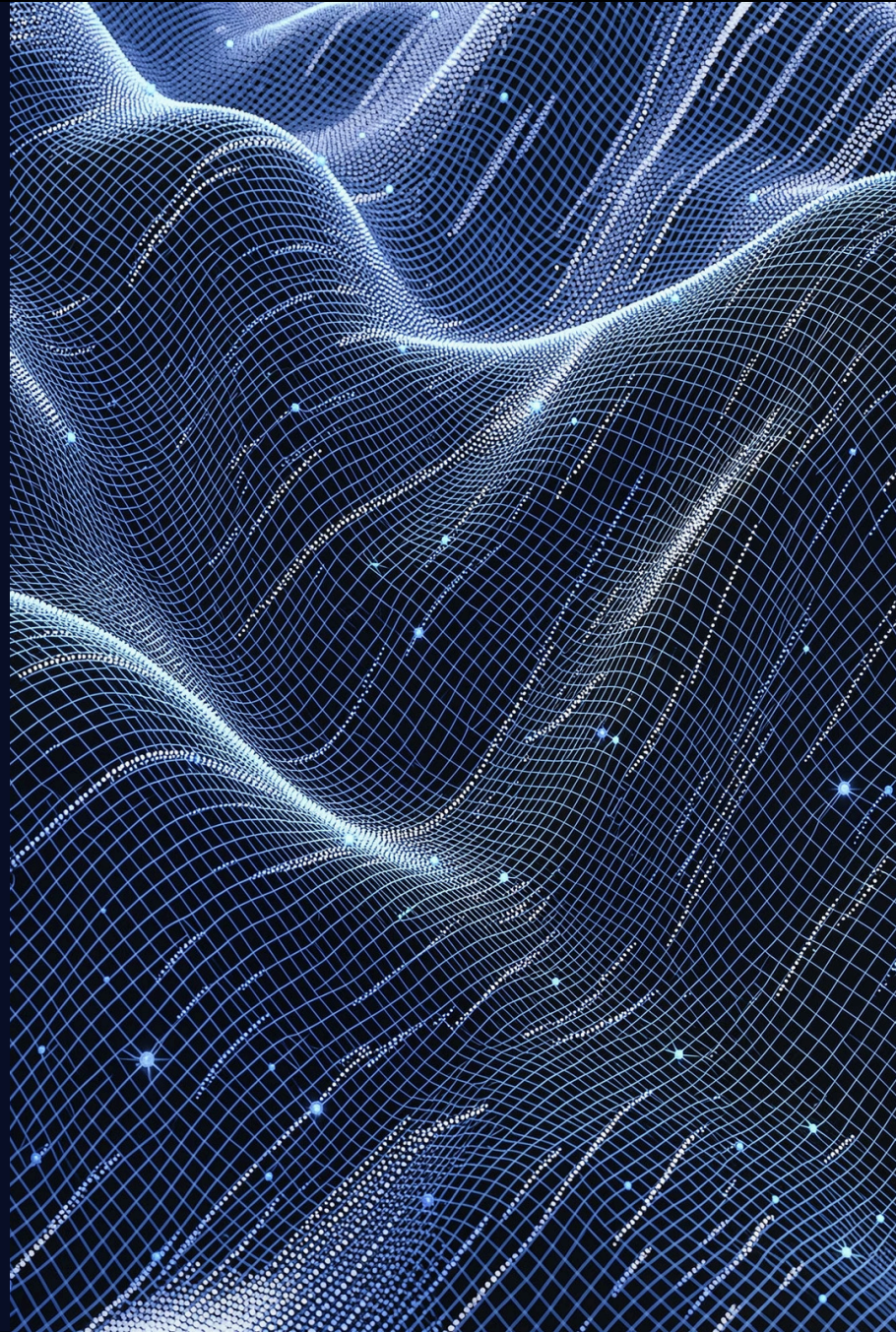


Table of Contents

01

Introduction

Phishing threat landscape & project scope

02

Motivation

Why deep learning for URL-based detection

03

DL and ML FLow

Dataset, preprocessing & tokenization

04

Feature Extraction

30 hand-crafted lexical features

05

Neural Network Architectures

5 DL families + 6 ML models

06

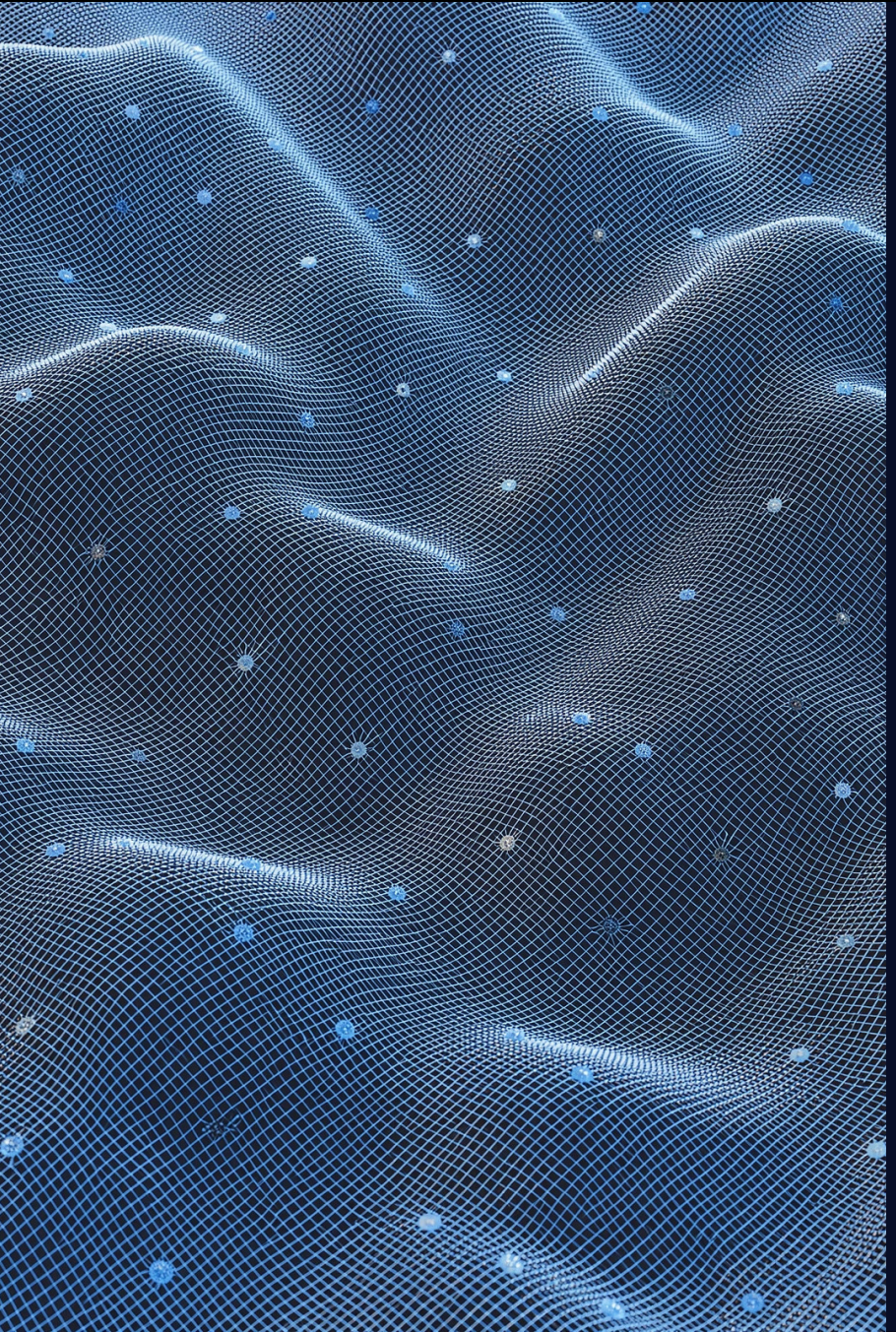
Results & Analysis

Performance comparison & key insights

07

Conclusion & Future Scope

Findings, limitations & next steps



Introduction

Phishing attacks deceive users by impersonating trusted entities via emails, fake websites, and malicious links. Traditional defenses – blacklists and heuristic rules – fail against zero-day campaigns.

Problem

Phishers exploit trust to steal credentials and sensitive data at scale.

Approach

Character-level deep learning pipeline – no external APIs or third-party lookups.

Scope

5 DL families (ANN, CNN, RNN, BiRNN, Attention) + 6 ML models compared.

Motivation

Why This Problem?

- Phishing caused **$4.1B in losses** in 2023 (FBI IC3 Report)
- Attacks surged **43%** in Q2 2022
- New malicious domains register every minute – blacklists cannot keep up
- URLs are the first attack vector; early blocking prevents the full chain

Why Deep Learning?

- Traditional ML (Random Forest, SVM) requires manual feature engineering
- DL models learn features **automatically** from raw character sequences
- Character-level models are language-independent and resilient to obfuscation
- CNNs detect local n-gram patterns; RNNs capture sequential structure

Methodology — Data Pipeline

DL flow

Raw URL → Tokenize → Pad (200) → Embed (50-D) → DL Model → Prediction

- Dataset: ~415,000 URLs (364K training + 51K testing), balanced between phishing and legitimate.
- Label Encoding: Phishing = 1 (positive class), Legitimate = 0.
- URL Normalization: Convert to lowercase; preserve the protocol prefix (http/https) as a feature.
- Character Tokenization: Each character is mapped to an integer index using a pre-built vocabulary.
- Sequence Padding: All URLs are padded or truncated to a fixed length of 200 characters.
- Embedding: Each character index is projected into a 50-dimensional continuous vector space.

ML Flow

Data Loading And Feature Extraction → Feature Vector Dataset → Model Training

Methodology — 30 Lexical Features

Structural Features

- URL length, domain length, path length
- Subdomain count, path depth
- Counts of: dots (.), hyphens (-), underscores (_), slashes (/)
- Counts of: @, ?, =, &, %, digits

Binary Flags (Security Indicators)

- has_ip_address – Domain is an IP (e.g., 192.168.1.1)
- has_port – Presence of non-standard port
- has_https / has_http – Protocol detection
- has_at_symbol – Presence of '@' (redirect trick)
- has_double_slash – Extra '//' in path
- has_dash_in_domain – Dashes mimic real domains
- is_shortened – URL shortening used (bit.ly, t.co)

Statistical Features

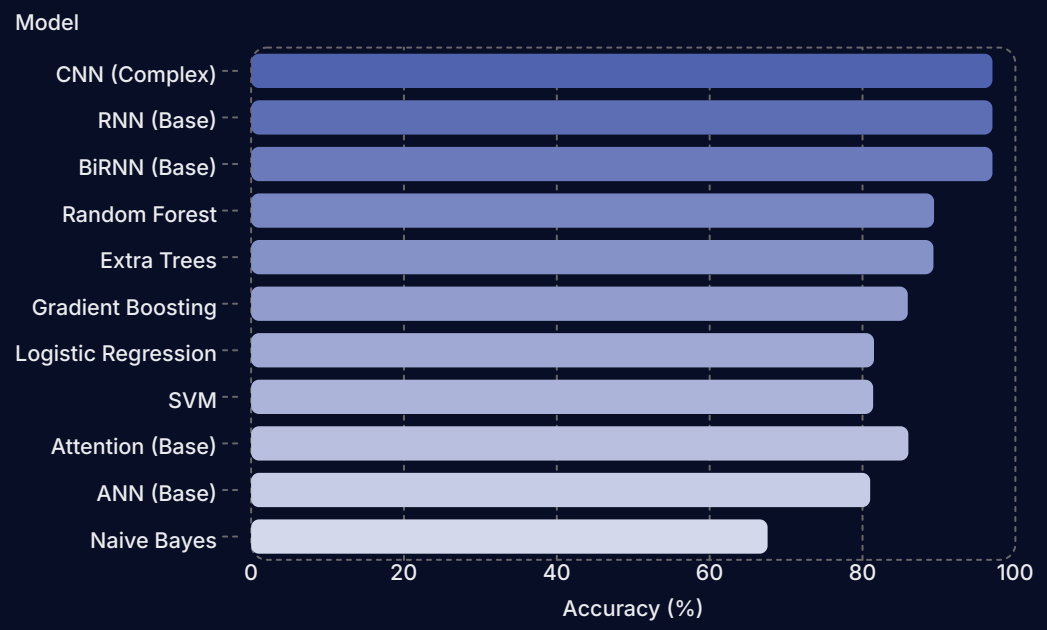
- Digit, letter, and special character ratios
- Shannon entropy of URL and domain (measures randomness)
- Character repetition count (detects keyword stuffing)
- Query parameter count

Neural Network Architectures

Model	Base Scenario	Complex Scenario
ANN	Embedding → Dense(128) → Flatten → Output	Embedding → 7× Dense(128, ReLU) → Flatten → Output
CNN	Embedding → Conv1D(128) → Flatten → Output	Embedding → 6 Conv Blocks (17 layers) → Flatten → Output
RNN	Embedding → LSTM(128) → Output	Embedding → 7× Stacked LSTM(128) → Output
BiRNN	Embedding → BiLSTM(128) → Output	Embedding → 6× BiLSTM(64) → BiLSTM(128) → Output
Attention	Embedding → Attention → Output	Embedding → LSTM + BiLSTM + 2× SelfAttn → Output

Training: Adam optimizer (LR = 0.01) · Binary Cross-Entropy loss · Batch size 1024 · Early stopping (patience = 3)

Results — Model Performance Comparison



Key Insights

- **CNN, RNN & BiRNN** lead at ~97% – pattern learning + sequence modeling
- **Random Forest** tops ML models at ~89.36%
- **Naive Bayes** underperforms (67.58%) due to strong independence assumptions
- DL outperforms ML via **automatic feature learning** from raw URLs

Results — Performance Analysis

Additional Observations

- Deep models show **better generalization** across diverse URL patterns
- High **precision (~0.98)** indicates very low false positives
- High **recall (~0.96)** ensures most phishing URLs are detected
- Balanced **F1-score (~0.97)** confirms overall robustness

Why Traditional ML Falls Short

- Relies on **manually engineered features**
- Cannot capture **sequential dependencies** in URLs
- Limited ability to detect **obfuscated or unseen patterns**

Performs well (~89%) but lacks adaptability compared to DL

Why CNN (Complex) Leads

- Lower layers detect local patterns: *"login"*, *"verify"*, *".exe"*
- Higher layers combine them into abstract phishing signatures
- Dropout (0.2) reduces overfitting; 17 layers learn richer representations

DL vs. Traditional ML

- ML depends on **handcrafted features**; best model (Random Forest) reaches ~89%
- DL achieves ~97% by learning features **automatically** from raw character sequences
- BiRNN matches CNN – bidirectional context improves sequence understanding

Scalability Insight

DL MODELS

High accuracy but **computationally intensive**

ML MODELS

Faster inference but **lower accuracy**

Conclusion & Future Scope

Conclusion

- Character-level DL achieves ~**97% accuracy**
- CNN & BiLSTM significantly outperform base models
- ML baselines useful but lower (~89%)

Limitations

- Static dataset – may not generalize to new trends
- Deep models computationally expensive for real-time use
- Analyzes URL string only, not page content or metadata

Future Scope

- Real-time detection system (latency <500ms)
- Chrome browser extension integration
- DNS/WHOIS lookup, typosquatting & obfuscation detection
- Full deployment: FastAPI backend + React dashboard

REFERENCES

- Sahingoz, O. K., Buber, E., Demir, O., & Diri, B.
  "Machine Learning Based Phishing Detection from URLs"
  Expert Systems with Applications, vol. 117, pp. 345–357, 2019
- DEPHIDES: *Deep Learning Based Phishing Detection System*
  IEEE Access, 2024
- Scikit-learn Documentation
  https://scikit-learn.org
- TensorFlow / Keras Documentation *(if used)*
  https://www.tensorflow.org
- NumPy Documentation
  https://numpy.org